Tutubi 1

Raffinamento dei requisiti

1. Utente registrato
    1.1. Cronologia dei video visionati
        1.1.1. Sequenza di tutti i video visionati, in ordine decrescente in base al loro istante di visualizzazione
        1.1.2. Le sue playlist (v. req. 6)
    1.2. nome (stringa)
    1.3. iscrizione (data e ora)
2. Video pubblicato
    2.1. Utente che l'ha pubblicato (v. req. 1)
    2.2. Titolo (stringa)
    2.3. Durata (ore, minuti, secondi)
    2.4. Descrizione (stringa)
    2.5. Nome della copia che viene salvata nel sistema (stringa)
    2.6. Categoria (unica) (v. req. 3)
    2.7. I suoi tag (v. req. 4) (almeno un tag??)
    2.8. Numero di visualizzazioni (intero>0)
    2.9. Se è una video risposta
        2.9.1. Vogliamo conoscere a quale video (di un altro utente) sta rispondendo
    2.10. Numero di video in risposta a questo video
    2.11. Se è stato censurato
        2.11.1. Motivo della censura (stringa)
        2.11.2. Istante in cui è stato censurato (data e ora)
3. Categoria
    3.1. Nome (unico) (stringa)
4. Tag
    4.1. Nome (unico) (stringa)
5. Commento
    5.1. Utente che ha lasciato commento (v. req. 1)
    5.2. istante di pubblicazione (data e ora)
    5.3. Video a cui è riferito (v. req. 2)
6. Playlist
    6.1. Utente proprietario (v. req. 1)
    6.2. Nome (stringa)
    6.3. Creazione (data ora)
    6.4. Elenco dei video (v. req 3)
    6.5. Tipo (pubblica o privata)

7. Funzionalità richieste al sistema
    7.1. Qualunque utente deve poter
        7.1.1. registrarsi (se non è ancora registrato)
        7.1.2. fare login (se è già registrato)
        7.1.3. condividere playlist personali pubbliche

7.1.4. guardare le playlist pubbliche di qualunque utente

7.1.5. fare ricerca dei video pubblicati

7.1.5.1. si vogliono restituire tutti i video disponibili di una categoria specificata che posseggono almeno uno tra i tag indicati, e che abbiano una valutazione media di almeno v}} (se un video non ha ancora alcuna valutazione, deve essere restituito comunque).

7.1.6. fare ricerca delle discussioni più animate al momento

7.1.6.1. Dopo aver specificato una categoria, verranno elencati i video col maggior numero (rispetto alla media??) di video in risposta

7.2. Utenti registrati possono

7.2.1. pubblicare video

7.2.2. pubblicare un video in risposta a quello di un altro utente

7.2.3. visualizzare video pubblicati

7.2.4. valutare video VISIONATO e pubblicato da un altro utente

7.2.4.1. Con un numero che va da 0 a 5

7.2.4.2. Al più una valutazione per ogni video visionato

7.2.5. lasciare commenti (solo testo) sui video VISIONATI

7.2.5.1. Può commentare diverse volte sullo stesso video

7.2.6. creare playlist personali (pubbliche o private)

7.2.7. modificare playlist personali

7.3. Tutubi deve poter

7.3.1. promuovere i video meglio valutati (v. req. 7.2.4) e scovare quelli con valutazioni basse (in che modo li può promuovere??)

7.3.2. censurare i video che violano le regole del servizio (copyright, oscenità, ecc)

7.3.2.1. Un video censurato non può essere né visionato, né votato, né commentato, né aggiunto ad alcuna playlist, né restituito come risultato di una ricerca.

7.3.2.2. Un video, una volta censurato, non può tornare più visibile

7.3.2.3. Il motivo di una censura deve essere mantenuto nel sistema per usi interni

Diagramma delle classi

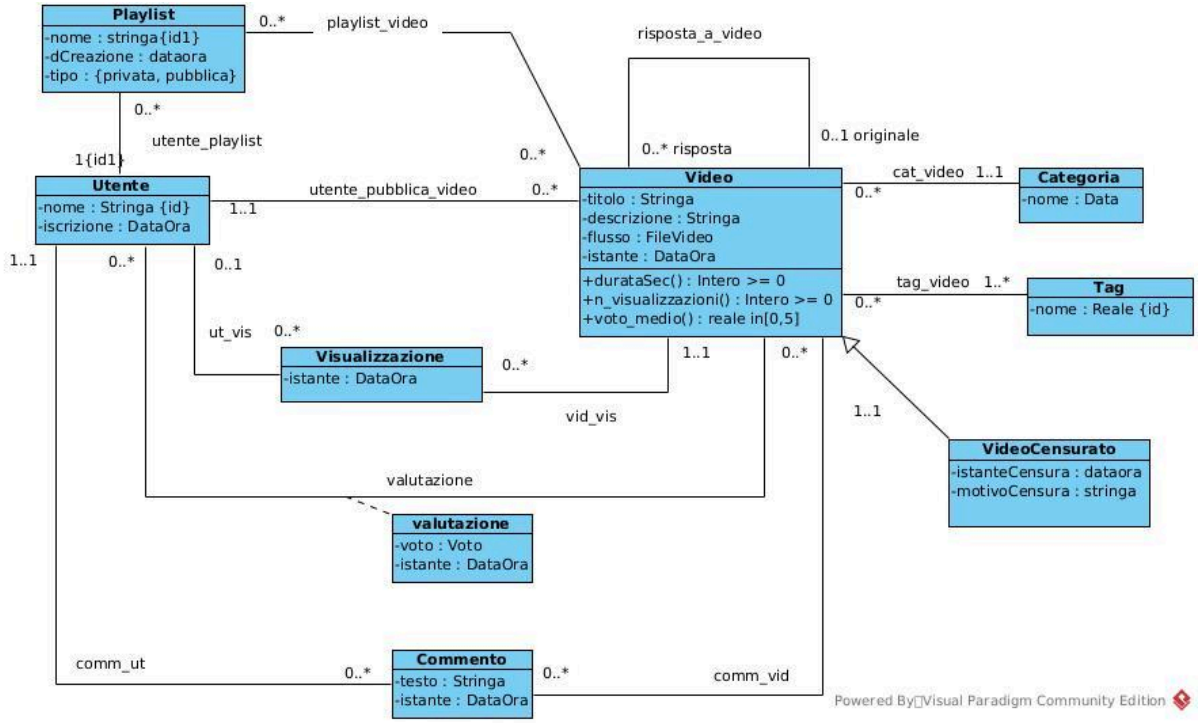
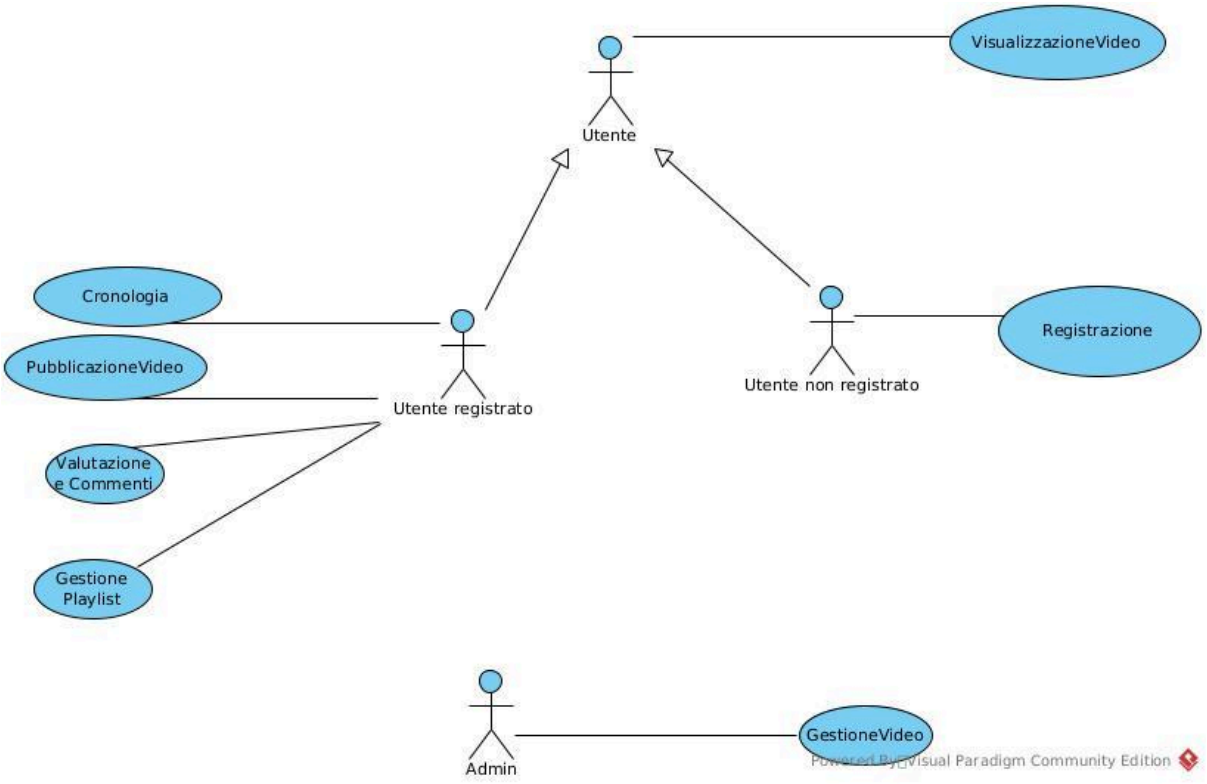


Diagramma use-case



Specifica dei nuovi tipi di dato

- FileVideo: sequenza di byte che codifica un flusso video
    - operazioni del tipo di dato
        - durataSec(f:FileVideo): Intero >= 0
            - pre: nessuna
            - post: result è la durata, in secondi, di “f”
- Voto : Intero in intervallo [0,5]
- DataOra:
    - ore: int>=0
    - minuti: intero in [0,59]
    - secondi: intero in [0,59]

Specifica dei vincoli e delle operazioni di classe

- **Nota**: In realtà mancano tanti vincoli temporali riguardo a quale azione avviene prima di un’altra, ma siccome sono banali e quasi identiche ho deciso di non specificarle tutte.
- Classe Visualizzazione
    - [V.Visualizzazione.non_censurato]
        - NOT EXISTS vis, i_vis, vid, i_cens: Visualizzazione(vis) and istante(vis, i_vis) and vid_vis(vid, vis) and VideoCensurato(vid) and istante(vid, i_cens) and i_cens < i_vis.
- Classe Utente
    - cronologia() : Cronologia
        - pre: nessuna
        - post:
            - Il livello estensionale dei dati viene modificato come segue
            - Sia V={ (video,v_ist) | Visualizzazione(v) AND istante(v, v_ist) AND utente_visualizza(this, v) AND visualizza_video(v, video) }
            - Sia V_sort un vettore che contiene tutti gli elementi di V, che però sono in ordine decrescente in base al valore del loro secondo elemento
            - **Nuove ennuple di predicati:**
                - Cronologia (V_sort)
            - return=V_sort

- Classe Video
    - [V.Video.autore_non_risponde_a_se_stesso]
        - ALL r,v,u : Video(r) AND Video(v) AND Utente(u)AND risposta_a_video(r,v) AND utente_pubblica_video(u,r)
          ->
          NOT utente_pubblica_video(u,v)
    - [V.Video_non_risponde_a_video_gia_censurato]

        - ALL r,v: Video(r) AND Video(v) AND risposta_a_video(r:risposta,v:originale) -> NOT VideoCensurato(v)
    - durataSec(): Intero >= 0
        - pre: nessuna
        - post: result è tale che
            - EXISTS f: flusso(this,f) AND durataSec(f,result)
    - n_visualizzazioni(): intero>=0
        - pre: nessuna
        - post:
            - Non ci sono modifiche sul livello estensionale dei dati
            - Sia V={ vs | Visualizzazione(vs) AND visualizza_video(v, this)} # Anche se Visualizzazione non ha nessun attributo ID ogni istanza di questa è sempre univocamente identificata
            - return = |V|
    - numRisposte(): intero>=0
        - pre: nessuna
        - post:
            - Non ci sono modifiche sul livello estensionale dei dati
            - Sia R={ r | Video(r) AND risposta_a_video(this: originale, r: risposta)}
            - return = |R|
    - voto_medio(): reale in [0,5]
        - pre:
            - EXISTS u: Utente(u) AND valutazione(u,this)
        - post:
            - Sia Voti = { (u,voto) | ALL : Utente(u) AND valutazione(u,this) AND voto(u,this,voto) }
            - result = ($SUM_{(u,voto) \text{ in } Voti}$voto) / |Voti|
- Classe Commento
    - [V.Commento.visualizzazione_pregressa]
        - ALL c, ic, u, v:
          Commento(c) and istante(c, ic) and comm_vid(c, v) and comm_ut(c, u)
          ->
          EXISTS vis,ivis:
          Visualizzazione(vis) and istante(vis, ivis) and ivis < ic and ut_vis(u, vis) and vid_vis(v, vis).

Altri vincoli esterni

- [V.videocensurato.no_visualizza]
    - ALL vis,u,v,istante_vis,istante_censura:
      Utente(u) AND VideoCensurato(v) AND Visualizazzione(vis) AND t_vis(u,vis) AND vid_vis(v,vis) AND istante(vis,istante_vis) AND istanteCensura(v, istante_censura) -> istante_vis < istante_censura
- [V.videocensurato.no_comments]
    - ALL u,v,comm,istante_comm:

Utente(u) AND VideoCensurato(v) AND istanteCensura(v, istante_censura) AND Commento(comm) AND comm_ut(comm,u) AND comm_vid(comm,v) AND istante(comm,istante_comm) -> istante_comm < istante_censura

- [V.videocensurato.no_valutazioni]
  - ALL u,v,istante_censura,istante_valutazione:
    Utente(u) AND VideoCensurato(v) AND istanteCensura(v, istante_censura) AND valutazione(u,v) AND istante(u,v,istante_valutazione) -> istante_valutazione < istante_censura
- [V.videocensurato.no_playlist]
  - #Non credo che convenga definire un vincolo per questo. Penso che verrà specificato nell'operazione di playlist che descrive l'inserimento di un video nella playlist
- [V.videocensurato.non_torna_visibile]
  - #Credo che basti NON definire nessun'operazione di use-case che permetta tale azione
- Associazione Valutazione
  - [V.valutazione.utente_ha_visualizzato_video]
    - ALL u,v,istante_valutazione:
      Utente(u) AND Video(v) AND valutazione(u,v) AND istante(u,v, istante_valutazione)
      ->
      EXISTS vis,istante_visualizzazione:
      Visualizzazione(vis) AND ut_vis(u,vis) AND vid_vis(v,vis) AND istante(u,vis, istante_visualizzazione) AND istante_valutazione > istante_visualizzazione
  - [V.valutazione.utente_non_autore]
    - ALL u,v:
      Utente(u) AND Video(v) AND valutazione(u,v)
      ->
      NOT utente_pubblica_video(u,v)
  - [V.valutazione.non_censurato]
    - NOT EXISTS u,v,istante_valutazione,istante_censura:
      Utente(u) AND VideoCensurato(v) AND valutazione(u,v) AND istante(u,v,istante_valutazione) AND istanteCensura(v,istante_censura) AND istante_valutazione>istante_censura
      #La mia soluzione NON considerava il fatto che i video censurati possono avere valutazioni fatte prima della censura

8. Funzionalità richieste al sistema
   - 8.1. Qualunque utente deve poter
     - 8.1.1. registrarsi (se non è ancora registrato)
     - 8.1.2. creare playlist personali (pubbliche o private)
     - 8.1.3. modificare playlist personali
     - 8.1.4. condividere playlist personali pubbliche
     - 8.1.5. guardare le playlist pubbliche di qualunque utente
     - 8.1.6. fare ricerca dei video pubblicati

8.1.6.1. si vogliono restituire tutti i video disponibili di una categoria specificata che posseggono almeno uno tra i tag indicati, e che abbiano una valutazione media di almeno v}} (se un video non ha ancora alcuna valutazione, deve essere restituito comunque).

8.1.7. fare ricerca delle discussioni più animate al momento

8.1.7.1. Dopo aver specificato una categoria, verranno elencati i video col maggior numero (rispetto alla media??) di video in risposta

8.2. Utenti registrati possono

8.2.1. pubblicare video

8.2.2. pubblicare un video in risposta a quello di un altro utente

8.2.3. visualizzare video pubblicati

8.2.4. valutare video VISIONATO e pubblicato da un altro utente

8.2.4.1. Con un numero che va da 0 a 5

8.2.4.2. Al più una valutazione per ogni video visionato

8.2.5. lasciare commenti (solo testo) sui video VISIONATI

8.2.5.1. Può commentare diverse volte sullo stesso video

8.3. Tutubi deve poter

8.3.1. promuovere i video meglio valutati (v. req. 7.2.4) e scovare quelli con valutazioni basse (in che modo li può promuovere??)

8.3.2. censurare i video che violano le regole del servizio (copyright, oscenità, ecc)

8.3.2.1. Un video censurato non può essere né visionato, né votato, né commentato, né aggiunto ad alcuna playlist, né restituito come risultato di una ricerca.

8.3.2.2. Un video, una volta censurato, non può tornare più visibile

8.3.2.3. Il motivo di una censura deve essere mantenuto nel sistema per usi interni

Specifica use-case VisualizzaVideo

- ricerca_video(cat: Categoria, tags: Tag[1..*], voto: Voto):Video[0..*]
  - pre: nessuna
  - post:
    - Non viene modificato il liv. estensionale dei dati
    - result = { v | Video(v) AND cat_video(cat,v) AND (EXISTS t: Tag(t) AND t in tags AND tag_video(t,v)) AND [($voto_medio_{Video}$(v,voto_medio) AND voto_medio>=v) OR (NOT EXISTS u: Utente(u) AND valutazione(u,v))]}

Specifica use-case Registrazione

- nuovo_utente(nome: Stringa): Utente
  - pre:
    - NOT EXISTS u: Utente(u) AND nome(u,nome)
  - post:

        - Il livello estensionale dei dati viene modificato
        - Nuovo elemento del dominio: alpha
        - Nuove ennuple di predicati:
            - Utente(alpha)
            - nome(alpha, nome)
            - iscrizione(alpha, adesso)
        - result = alpha

Specifica use-case GestionePlaylist

- nuova_playlist(nome: stringa, tipo: {pubblica,privata}): Playlist
    - pre:
        - Sia u l'utente che ha invocato questa operazione
        - Non esistono altre playlist dell'utente "u" con lo stesso nome di quello che voglio creare, ovvero…
            - NOT EXISTS p: Playlist(p) AND utente_playlist(u,p) AND nome(p,nome)
    - post:
        - Sia u l'utente che ha invocato questa operazione
        - Il livello estensionale dei dati viene cambiato
        - Nuovo elemento del dominio: alpha
        - Nuove ennuple di predicati:
            - Playlist(alpha)
            - nome(alpha, nome)
            - dCreazione(alpha, adesso)
            - tipo(alpha, tipo)
            - utente_playlist(u,alpha)
        - result = alpha